Embedded Data Structure
(Lecture 01)

Introduction to Data Structure

2025. 3. 5

Prof. Wonjun Kim

School of Electrical and Electronics Engineering

Who am I

Prof. Wonjun Kim

Engineering Building C, 424-2

E-mail : wonjkim@konkuk.ac.kr

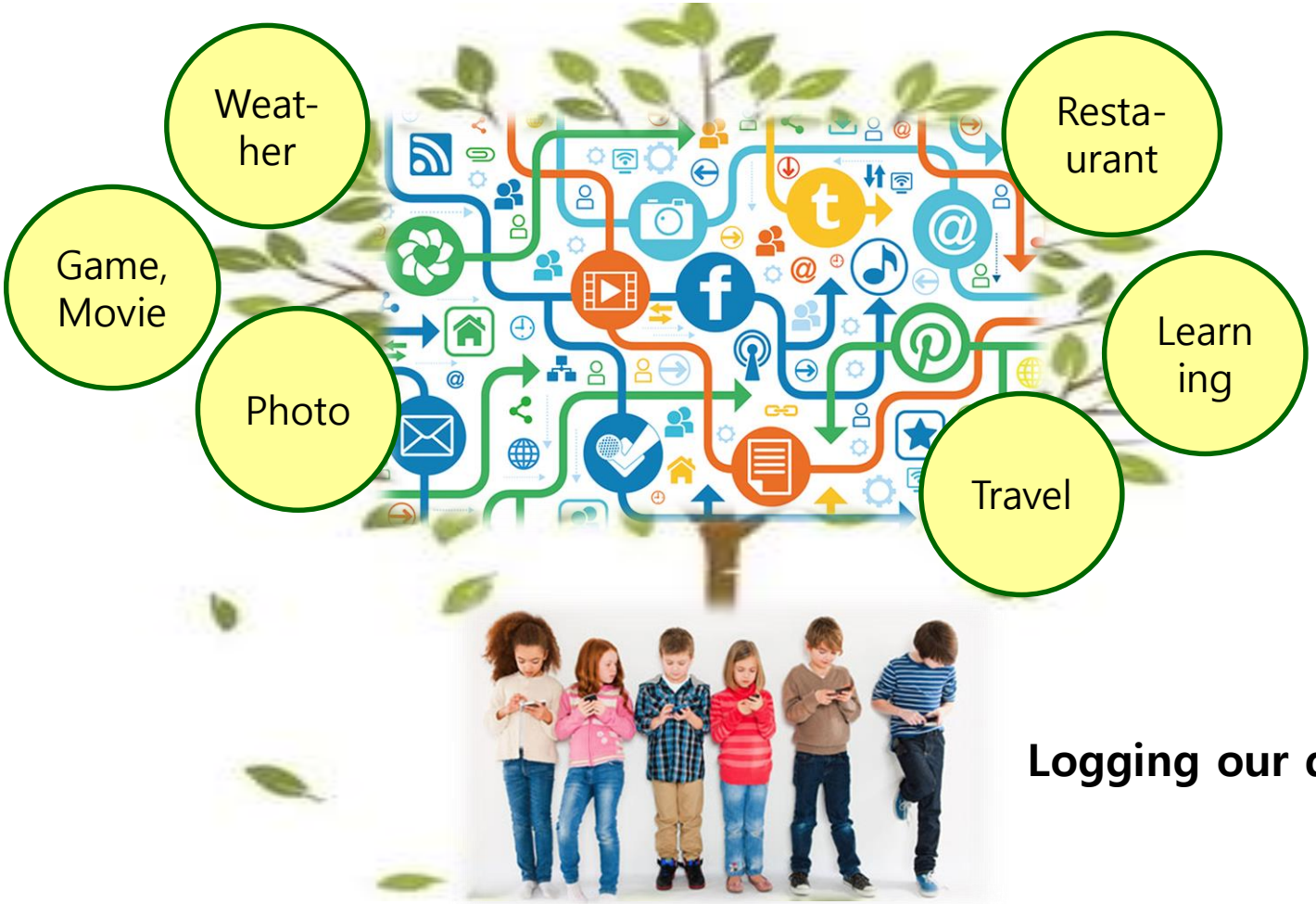
Research Interest :

Computer vision, image/video processing, pattern recognition, machine learning

※ Lab. homepage : **https://sites.google.com/view/dcvl**

Before Starting … (1/4)

- **Revolution of data !**



Logging our daily life

In our daily life

Before Starting … (2/4)

- **Revolution of data !**
  - **Data is King at Amazon – Ronny Kohavi (前 Amazon director)**

ChatGPT

+

Transformer

- ✓ >175 billion parameters
- ✓ Autoregressive learning

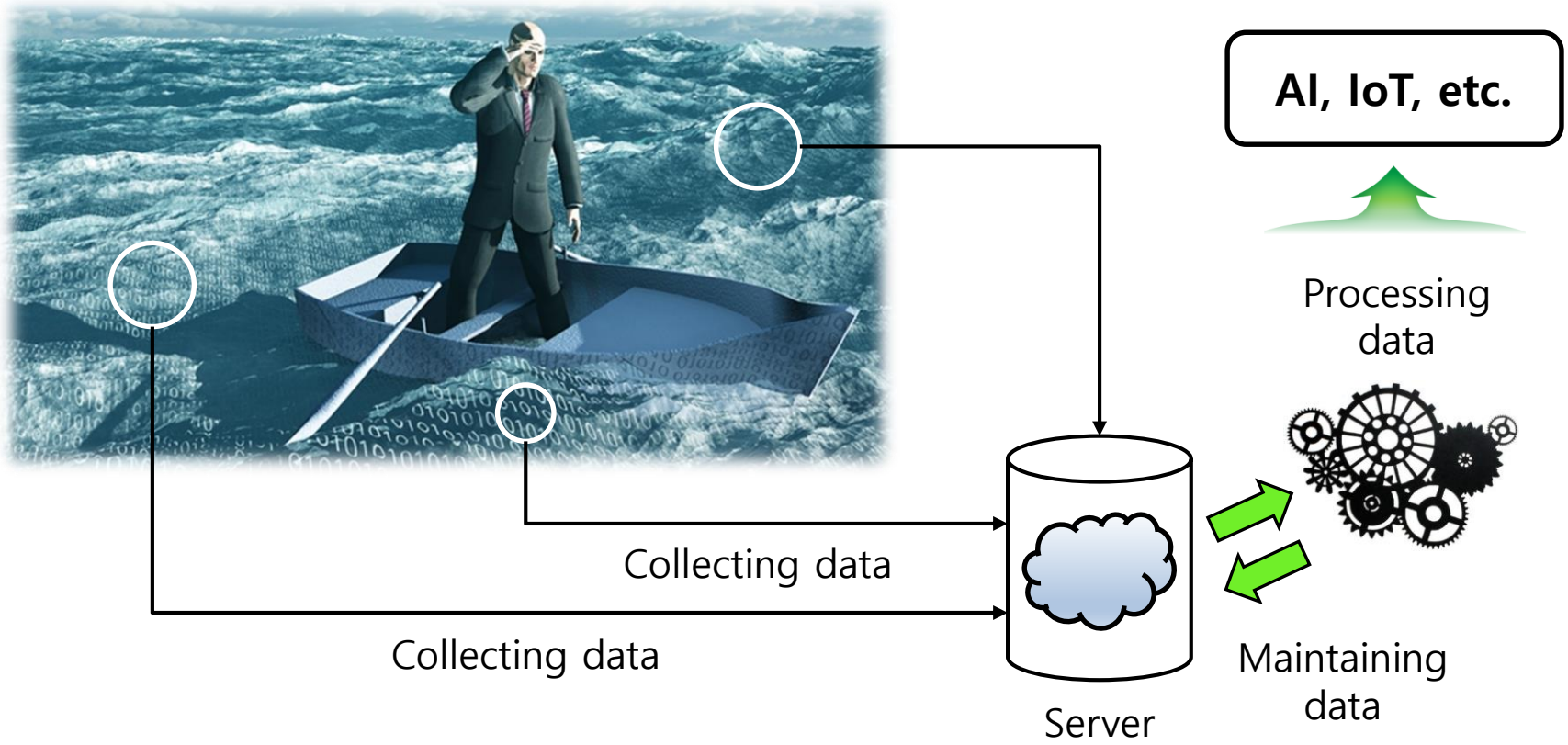
Hyper-scale data

- ✓ >570GB text from wiki, blog, webpage, etc.
- ✓ 10 thousand inspectors

➔ **We need to be prepared to handle big data !**

Before Starting … (3/4)

- **Key point : finding the useful "meaning" in data**
  - **To do this, handling and managing data is very important**



Before Starting … (4/4)

- **Data, data, and data (of course, high-quality)**
  - **Data is food for AI !**

    : Instead of emphasizing on tuning models, clean the data as much as possible

Model-centric AI → Data-centric AI

(https://www.youtube.com/watch?v=06-AZXmwHjo)

Class Schedule

- **What we learn in this class …**
  - **Introduction to data structure**
  - **Revisit C programming (array, pointer, memory, etc.)**
  - **Linked list**
  - **Stack and queue**
  - **Data labeling (by using stack)**
  - **Tree structure**
  - **Sorting algorithms**
  - **Searching algorithms**
  - **Practical examples**

Lecture Note

※ Language : C or (C++)

KU KONKUK UNIVERSITY

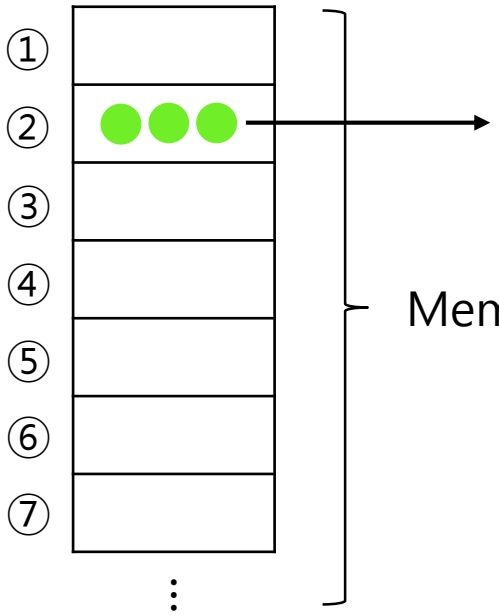
▪ Data by C programming

- **Why C (vs. Matlab, Python, etc.) ?**

  * C is the only one that handles the address of the memory

  → Efficient to access the memory

  → Critical to the embedded applications



How can we copy (or move) this data to ⑤ ?

: In C, ( ) can make it possible !

Memory structure is linear !

※ Difference btw. global and local allocation

Introduction (2/8)

KU KONKUK UNIVERSITY

▪ Why C for embedded programming ? (summary)

- **Two reasons : memory and hardware control**

1) Only C performs in an efficient way under limited memory environments

: Other visual languages require quite a lot memory spaces

※ The capacity of program executable binary is very small in C

2) C can efficiently assign values in a specific address using "pointer"

: Data can be easily handled with hardware controls

➡ **That's the reason why we focus on "pointer" in C programming**

➡ **For efficiently managing the memory space,**
memory assign/free needs to be conducted with pointers
(except for the global data)

Introduction (3/8)

KU KONKUK UNIVERSITY

▪ How about big data ?

- **Problem of memory allocation (> 10 GB)**

  ※ Find the memory limit for Visual Studio 2020 Community (64bit)

  Alternatives :

  1) Divide the data and use the multi-core processor

     : Be careful for handling data seamlessly

  2) Via GPU (e.g., RTX 4090 (24GB memory))

     : With CUDA coding → efficiency ↑ (c.f. deep learning scheme)

- **Monitoring the working memory**

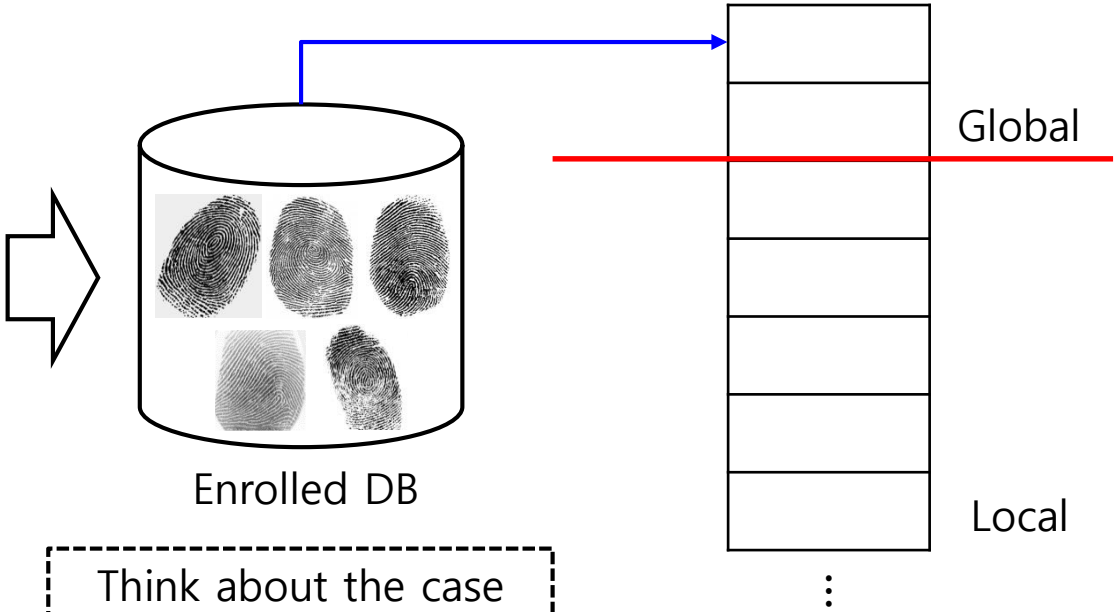
  : To prevent the stack overflow, allocation-free needs to be fully conducted

  ← It will be helpful to handle big data in your systems

- **Example : embedded system**
  - **Example of data handling : fingerprint recognition**

※ Difference between global and local allocation (data store)

[Fingerprint recognition]



Think about the case of turn-off

- **Example : embedded system – cont'd**
  - **Efficient use of memory is very important !**

※ Data needs to be clearly structured and maintained for efficiency

[Think about the space]

1	9
2	34
3	15
4	-
5	99
6	-
7	45
⋮	

before

1	9
2	-
3	15
4	-
5	-
6	-
7	45
⋮	

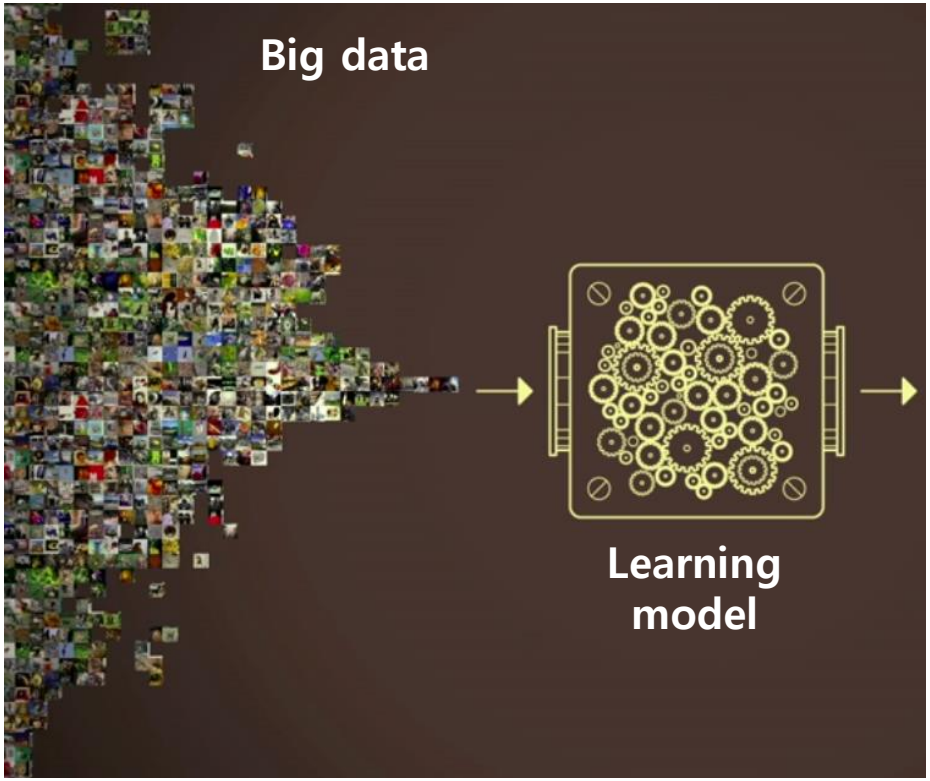
after

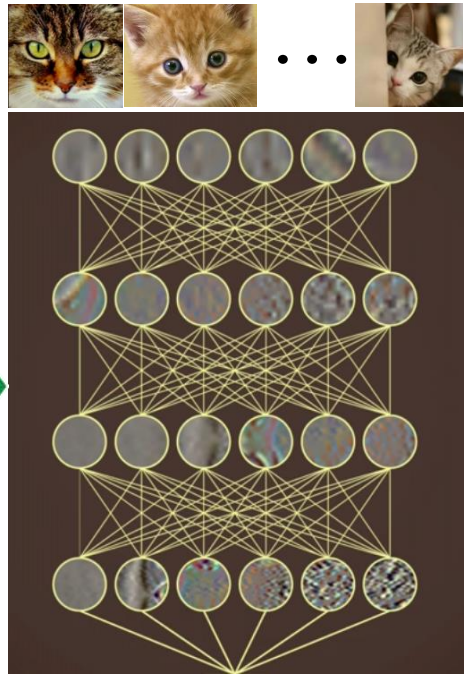
[Think about the memory space]

Introduction (6/8)

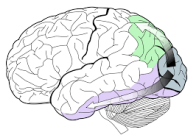
- **Example : deep learning with big data**
  - **Example of data load and processing in GPU**

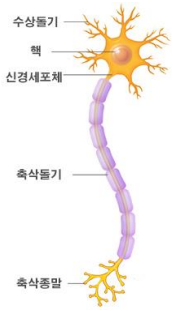
※ This scheme is the heart of the field of deep learning





"Cat"





KU KONKUK UNIVERSITY

▪ Communication between CPU and GPU

- **Design carefully training scheme**

→ This is because data transferring makes a significant bottleneck !

SSD (Data is here)

CPU
(Intel® Core i7 and Xeon® processor E5-2600, E5-1600 v3 v4)

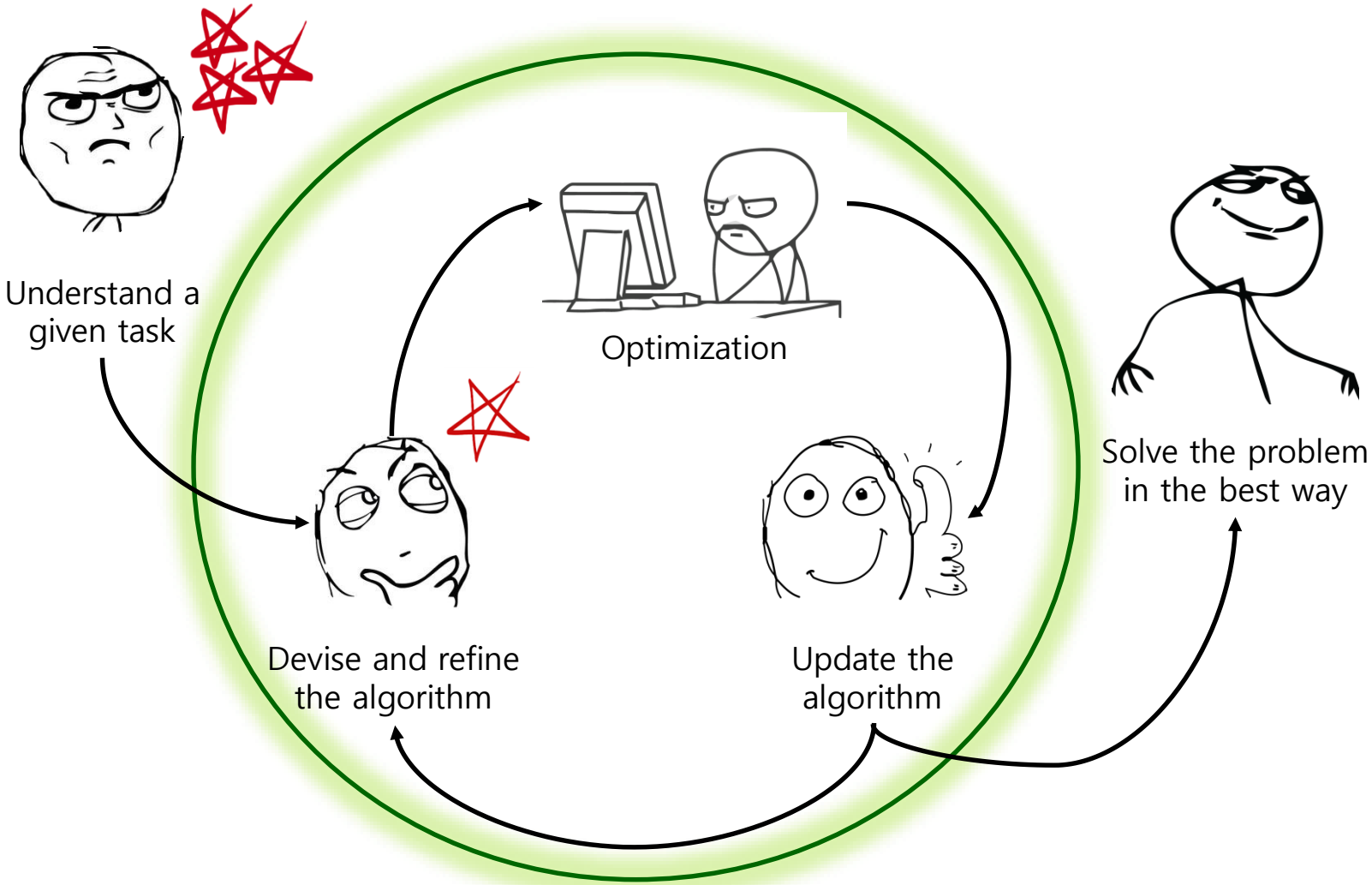
4 GPUs (Model is here)

Read all the data into RAM in GPUs

[NVIDIA SDX DevBox (RTX 3090Ti) @ our lab.]

Introduction (8/8)

- **From data to algorithm :**



Warming-up Example (1/3)

- **Data access (with memory usage)**
  - **Throw a dice 100 times and compute the frequency (빈도수)**

※ Use the rand() for generating the random number

Make your C codes !

Warming-up Example (2/3)

- **Data access – cont'd**
  - **Implementation details**

1) By using the "scanf", input your trial numbers (e.g., 100)
2) By using the "rand()", you can get the random variable from 1 to 6
3) By using the "for" (or whilie) loop, you can get a number at each iteration
4) Store this number in your memory !
5) Allocate the memory space to store the frequency as follows :

e.g.) 100 trials → freq[1] = 30, freq[2] = 35, freq[3] = 9
freq[4] = 10, freq[5] = 9, freq[6] = 7

※ You need to use "malloc" or "calloc" for allocating the memory (freq)

Warming-up Example (3/3)

- **Appendix : time checker**
  - **Compute the processing time as follows :**

```
#include <Windows.h>

LARGE_INTEGER freq, start, stop;
double diff;

QueryPerformanceFrequency(&freq); // computer frequency
QueryPerformanceCounter(&start); // starting point

Algorithm

QueryPerformanceCounter(&stop); // stopping point
diff = (double)(stop.QuadPart - start.QuadPart)/ freq.QuadPart;
```

Measuring time

Summary

- **Data revolution**
  - **Handling data (with C) is very important for real-world applications**
- **Enjoy thinking with your programming**